

Effort Estimation

Effort Estimation in Software Development is the process of predicting the amount of work and time required to complete specific tasks or a project. It involves estimating the number of person-hours, days, or weeks needed to design, code, test, and deliver software. Accurate effort estimation is critical for planning, budgeting, and resource allocation in software projects.

Effort estimation is the process of estimating the **amount of work required to develop a software project**.

It tells us **how much human effort** is needed to complete the project. Effort is usually measured in **person-hours, person-days, or person-months**.

Example

Suppose a project requires **600 hours** of work.

If one developer works 8 hours per day:

or

This does **not necessarily mean** one person must work for 75 days. Four developers could potentially complete the work in a shorter calendar time, although adding people does not always reduce time proportionally.

Effort estimation is essential for several reasons:

Project Planning: It helps define the scope, schedule, and milestones of a project.

Resource Allocation: Ensures that the right amount of resources are assigned to each task to avoid under- or over-utilization.

Budgeting: Provides a basis for estimating costs and controlling expenses throughout the project.

Risk Management: Helps identify potential risks related to time and resource constraints, allowing for early mitigation strategies.

Several techniques are commonly used for effort estimation in software development:

1. **Expert Judgment:** Relies on the experience of senior developers or project managers who have worked on similar projects to provide estimates.
2. **Analogous Estimation:** Uses historical data from past projects that are similar in scope and complexity to estimate the effort required for the current project.
3. **Parametric Estimation:** Involves using mathematical models to estimate effort based on project parameters, such as the number of lines of code (LOC), function points, or user stories.
4. **Delphi Method:** A group of experts provides estimates independently, and then the results are discussed collectively to reach a consensus.
5. **Wideband Delphi:** A variation of the Delphi method, where a group of experts discusses and revises their estimates in several rounds until a consensus is reached.

6. **Three-Point Estimation:** Involves estimating the best-case (optimistic), worst-case (pessimistic), and most likely scenarios, then calculating an average or weighted estimate.
7. **Use Case Points:** Estimates effort based on the number and complexity of use cases in the software.

To perform effort estimation, follow these steps:

1. **Break Down the Project:** Decompose the project into smaller tasks or deliverables, such as features, modules, or components.
2. **Choose an Estimation Technique:** Select an appropriate estimation technique based on the project's nature and available data.
3. **Gather Data:** Collect relevant data, such as historical information, complexity factors, and team capacity.
4. **Estimate Each Task:** Apply the chosen technique to estimate the effort required for each task or deliverable.
5. **Aggregate Estimates:** Sum the individual task estimates to obtain the total project effort.
6. **Review and Refine:** Review the estimates with stakeholders and refine them based on feedback and additional information.

Common challenges include:

1. **Uncertainty and Ambiguity:** Incomplete or unclear requirements can lead to inaccurate estimates.
2. **Underestimation:** Developers often underestimate effort due to optimism bias, leading to missed deadlines.
3. **Overestimation:** Overestimating effort can result in resource overallocation and increased project costs.
4. **Changing Requirements:** Frequent changes in project scope or requirements can render initial estimates inaccurate.
5. **Skill Variability:** Differences in team members' skills and experience can affect the accuracy of effort estimates.

To improve the accuracy of effort estimation:

Involve Experienced Team Members: Engage developers and project managers with relevant experience in the estimation process.

1. **Use Historical Data:** Leverage data from previous projects with similar characteristics to inform estimates.
2. **Refine Requirements:** Work closely with stakeholders to clarify requirements and reduce ambiguity.
3. **Iterative Estimation:** Update estimates as more information becomes available, especially in Agile environments.
4. **Review Past Estimates:** Analyze the accuracy of past estimates and learn from any discrepancies to improve future estimations.

Several tools can assist with effort estimation:

1. **Project Management Software:** Tools like JIRA, Microsoft Project, and Trello offer features for tracking and estimating effort.
2. **Estimation Tools:** Specialized tools like Function Point Analysis (FPA) calculators, COCOMO (Constructive Cost Model), and PERT (Program Evaluation Review Technique) charts can help with more sophisticated estimations.
3. **Spreadsheets:** Excel or Google Sheets can be used to create custom estimation models and track estimates.

Estimation Models in Software Engineering

An **estimation model** is a mathematical or systematic method used to estimate the **effort, cost, development time, and resources** required for a software project.

Major Software Estimation Models

1. **COCOMO Model**
 - Developed by **Barry Boehm**.
 - Uses the size of software, usually **KLOC (thousand lines of code)**.
 - Estimates effort, development time, and staffing.
 - Example formula:

Meaning of each term

- **Effort** = Amount of work required, usually measured in **person-months (PM)**.
- **KLOC = Kilo Lines of Code**, meaning thousand lines of source code.
 - 1 KLOC = 1,000 lines of code.
- **a** = A constant that depends on the **type of software project**.
- **b** = An exponent that also depends on the **type of project**.
- **^ab** means **raised to the power b**.

Example

Suppose:

- KLOC = 10
- a=2.4
- b=1.05

Then: $\text{Effort} = 2.4(10)^{1.05}$

Effort $\approx$ 26.9 person-months

So, the estimated effort is approximately **27 person-months**.

2. **Function Point Model**
 - Estimates software size based on the **functionality provided to the user**.
 - Considers:
 - Inputs
 - Outputs
 - User inquiries
 - Internal files

- External interfaces
- 3. **Putnam Model**
 - Based on the **software life-cycle and manpower distribution**.
 - Uses the relationship between software size, effort, and development time.
- 4. **Expert Judgment Model**
 - Experienced software professionals estimate effort and cost.
 - Useful when historical data or accurate mathematical information is not available.
- 5. **Analogy-Based Estimation**
 - The new project is compared with **similar completed projects**.
 - The effort of the previous project is adjusted according to differences in size and complexity.

Simple Example

Suppose a previous project of similar complexity required **10 person-months**. A new project is estimated to be about **20% larger**.

Approximate effort:

In short

Estimation model → Software size → Effort → Cost → Development time → Resources

So, the estimated effort is approximately **27 person-months**.

Why is KLOC raised to power b?

The important idea is that **effort does not always increase directly in proportion to software size**.

For example, if software size doubles, the effort may increase by **more than double** because larger software usually involves:

- More complexity
- More communication between developers
- More testing
- More integration
- More management

Therefore, the exponent **b** is used to represent this relationship.

Remember

E → Effort

a → Constant

KLOC → Software size

b → Exponent